

AdminForge: Gestão de Identidades Privilegiadas em Frotas de Servidores Linux

Modelagem da Primeira Versão

Anonymous Authors

Abril de 2026

Resumo

Este documento descreve a primeira versão do *AdminForge*, um sistema de linha de comando para gerenciar contas de administradores em cerca de seiscentos servidores Linux. O texto apresenta o modelo de dados, a arquitetura, os princípios de design (SOLID e KISS), os casos de uso detalhados com histórias de usuário, os fluxos principais e os cuidados de segurança. A proposta é simples: evitar ferramentas pesadas como FreeIPA ou LDAP, usar poucas dependências, manter um histórico completo do que o operador faz e oferecer auditoria operacional sob demanda para inspecionar usuários e serviços presentes em cada servidor. Como o sistema é usado de forma pontual, muitas decisões de arquitetura ficam mais simples.

Sumário

1	Contexto	2
1.1	Superadmin e admin	2
1.2	Princípios	2
2	Modelo de dados	3
2.1	Regra central	3
2.2	Diagrama de entidades	3
2.3	Descrição de cada entidade	4
3	Arquitetura	5
3.1	Persistência	5
4	Princípios de design	6
4.1	KISS	6
4.2	SOLID	6
4.3	Padrões aplicados	7
5	Casos de uso	7
5.1	UC-1: Cadastrar admin	7
5.2	UC-2: Cadastrar chave SSH	8
5.3	UC-3: Gerenciar grupo de admin	9
5.4	UC-4: Cadastrar servidor	9
5.5	UC-5: Gerenciar grupo de servidor	10
5.6	UC-6: Conceder ou revogar acesso	10
5.7	UC-7: Ver prévia	11
5.8	UC-8: Aplicar mudanças	11

5.9	UC-9: Ver histórico	12
5.10	UC-10: Auditar usuários e serviços do servidor	13
6	Fluxos principais	13
6.1	Cadastros (mudar o estado desejado)	14
6.2	Aplicar mudanças	14
6.3	Inspeção operacional	14
6.4	Desligamento	15
7	Cuidados de segurança	15
8	Próximos passos	16
8.1	Questões em aberto para a reunião	16

1 Contexto

O CPD tem cerca de seiscentos servidores Linux e vinte e poucos administradores que precisam ter conta neles. Cada admin tem acesso a um grupo diferente de servidores, e as equipes mudam com o tempo. Hoje, tudo é feito manualmente: quando entra alguém novo, o acesso é configurado servidor por servidor; quando alguém sai, o acesso deveria ser removido, mas é comum ficar conta ativa esquecida.

Existem ferramentas prontas pra isso, como FreeIPA ou OpenLDAP com Kerberos, mas elas trazem muita infraestrutura e manutenção pro nosso caso. A ideia é construir algo mais simples: um sistema de linha de comando que resolva o problema sem virar ele próprio um ponto único de falha.

1.1 Superadmin e admin

Dois papéis aparecem no texto todo e convém fixá-los agora:

Superadmin. A pessoa (ou pequeno grupo) que opera o AdminForge. Roda a CLI, cadastra admins, aplica mudanças. É o único ator do sistema.

Admin. O administrador gerenciado: a pessoa cuja chave SSH o sistema instala nos servidores. Não opera o AdminForge; do ponto de vista do sistema, é um dado.

A distinção importa na auditoria: o AdminForge registra o que o Superadmin faz, não o que os admins fazem depois de logar nos servidores (isso é papel do `sshd` e do `syslog` de cada máquina). Em contrapartida, o sistema pode fazer *inspeção operacional* sob demanda para listar usuários existentes e serviços em execução num servidor, ajudando a detectar incoerências como um usuário `tomcat` presente sem nenhuma instância de Tomcat rodando.

1.2 Princípios

- (i) **Modular.** Partes com responsabilidades claras, cada uma substituível sem mexer nas outras.
- (ii) **Independente em tempo de uso.** Uma vez aplicada a configuração, cada servidor funciona sozinho. Se o AdminForge sair do ar, ninguém perde acesso.
- (iii) **Só CLI.** Sem interface gráfica na v1.
- (iv) **Registra tudo, com resultado.** Cada ação do Superadmin fica no histórico, com status final (sucesso, falha, sucesso parcial).
- (v) **Só aplica o que mudou.** O sistema sabe o que já está em cada servidor. Se um admin novo entra num grupo de cinco, só a chave dele é propagada.

(vi) **Inspeciona o real sob demanda.** Quando preciso, o Superadmin consulta via SSH os usuários existentes e os serviços ativos de um servidor para comparar configuração declarada com a realidade operacional.

2 Modelo de dados

São sete entidades. Três conceitos compõem o domínio central (admin, servidor e os grupos que conectam os dois); um conecta grupos (permissão); um guarda as chaves (credencial); um registra o que o Superadmin faz (histórico).

2.1 Regra central

Um admin só acessa um servidor se ambos estiverem em grupos ligados por uma permissão. Não existe ligação direta admin–servidor. Disso saem duas consequências:

- Um admin pode ser cadastrado sem grupo. Fica registrado, mas sem acesso a nada.
- Um servidor pode ser cadastrado sem grupo. Fica registrado, mas ninguém o acessa via Admin-Forge.

2.2 Diagrama de entidades

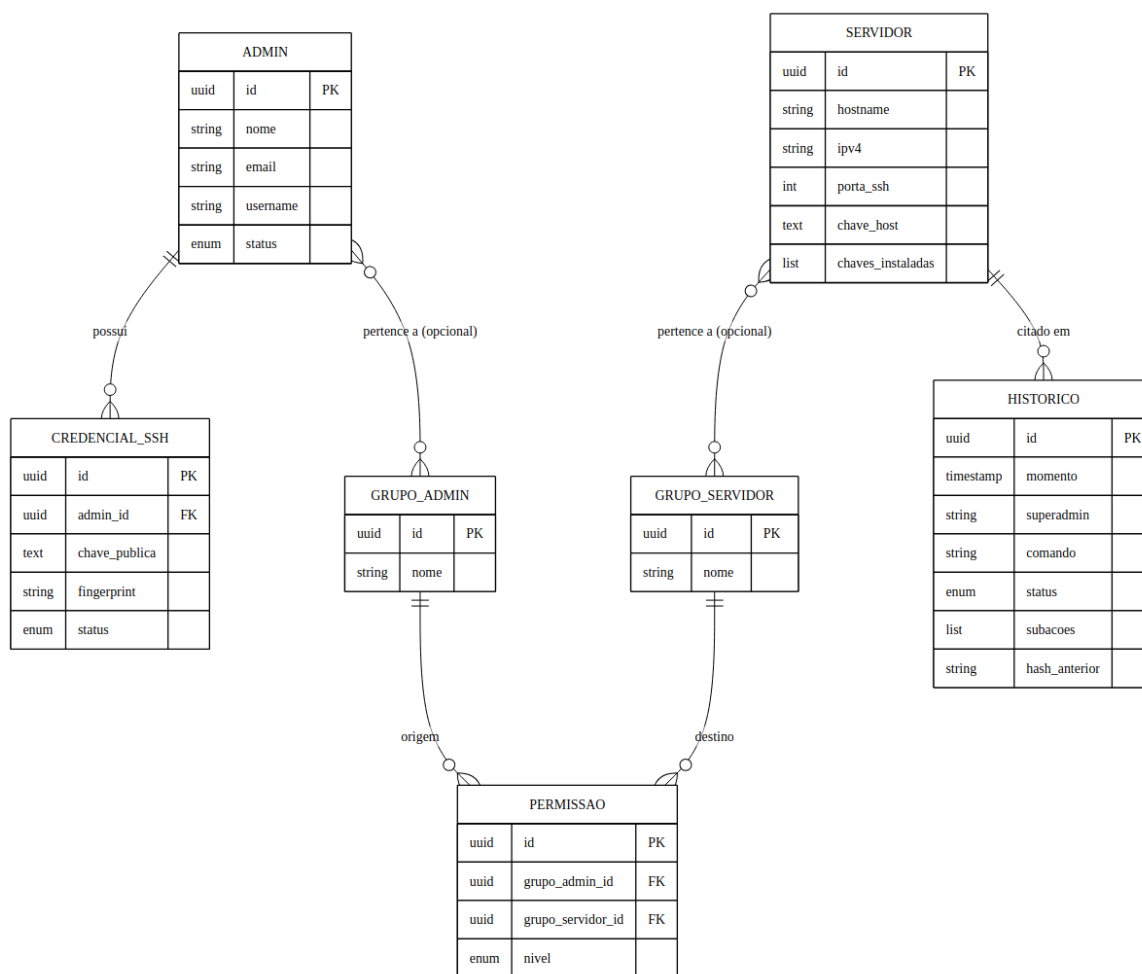


Figura 1: Modelo de entidades. Admin e Servidor podem existir sem grupo. O Servidor guarda em `chaves_instaladas` o que está de fato nele agora. O Histórico registra cada comando do Superadmin; quando o comando gera ações em servidores, elas ficam na lista `subacoes`.

2.3 Descrição de cada entidade

2.3.1 Admin. A pessoa gerenciada. Tem nome, email, *username* e status (ativo, inativo, bloqueado). Pode existir sem grupo.

2.3.2 Credencial SSH. Chave pública de um admin. Um admin pode ter mais de uma chave; cada uma pode ser ativa ou revogada.

2.3.3 Grupo de Admin. Junta admins com função parecida: *sysadmins*, *dba*, *rede*.

2.3.4 Servidor. Máquina Linux gerenciada. Guarda endereço, *host key* e também *chaves_instaladas*: a lista das credenciais que estão de fato instaladas nele agora. É isso que permite ao sistema calcular o delta e aplicar só o que mudou. Também pode ser alvo de inspeção operacional sob demanda, para listar usuários locais e serviços ativos. Pode existir sem grupo.

2.3.5 Grupo de Servidor. Junta servidores: *produção*, *homologação*, *bancos*.

2.3.6 Permissão. Liga um Grupo de Admin a um Grupo de Servidor, com nível *shell* ou *sudo*.

2.3.7 Histórico. Cada comando do Superadmin na CLI vira uma entrada. Guarda quem executou, quando, o comando, o status final (*sucesso*, *falha*, *sucesso parcial*) e, se o comando gerou ações em servidores, uma lista de subações (servidor, ação, status, mensagem de erro). Cada entrada carrega o *hash* da anterior, formando uma corrente verificável.

Um exemplo concreto de entrada do Histórico, correspondente a um *apply* em que um dos servidores estava fora do ar:

```
id: OP-0042
momento: 2026-04-22T14:32:11
superadmin: admin0001
comando: apply
status: sucesso_parcial          # 2 de 3 subacoes deram certo
subacoes:
- servidor: web-01
  acao: adicionar_chave
  credencial: marina:ed25519-abc
  status: sucesso
- servidor: web-02
  acao: adicionar_chave
  credencial: marina:ed25519-abc
  status: sucesso
- servidor: db-03
  acao: adicionar_chave
  credencial: marina:ed25519-abc
  status: falha
  erro: "ssh: connect timeout after 30s"
hash_anterior: 7c4a8d09...
```

Esse registro responde à pergunta “o que falhou?”. O Superadmin consulta via `adminforge history show OP-0042`, que imprime uma tabela das subações:

```
$ adminforge history show OP-0042

OP-0042  2026-04-22 14:32  admin0001  apply  SUCESSO PARCIAL (2/3)

web-01  adicionar marina  OK
web-02  adicionar marina  OK
db-03   adicionar marina  FALHA  ssh: connect timeout after 30s
```

A subação em *falha* continua no delta calculado pelo Planner; no próximo *apply*, só ela é retentada (as duas de sucesso não são reexecutadas, pois já constam em *chaves_instaladas*).

Por que *chaves_instaladas* no Servidor. É a forma mais simples de responder “o que já está *deployado*”. Se o grupo de admin tem cinco membros e entra o sexto, o sistema compara o desejado (seis chaves) com o que está em cada servidor (cinco chaves) e só aplica a diferença. Sem esse campo, teríamos que reconectar em todos os servidores a cada *apply* ou refazer o trabalho.

Inspeção não vira fonte da verdade. A listagem de usuários e serviços do servidor é uma leitura pontual via SSH. Ela serve para auditoria operacional e diagnóstico, não para substituir o estado declarado no Store nem o campo *chaves_instaladas*. Exemplo: encontrar o usuário *tomcat* sem nenhum serviço Tomcat ativo pode indicar sobra operacional, mas não altera nada automaticamente.

3 Arquitetura

Seis componentes, cada um com uma função clara. A CLI é a única porta de entrada; os demais são internos.

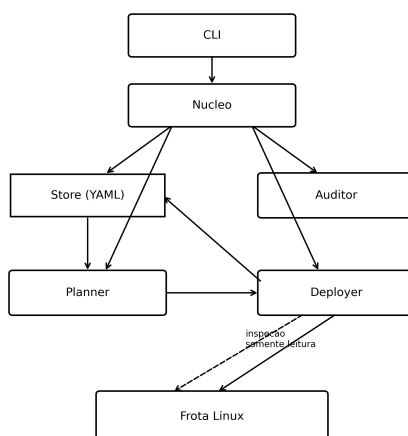


Figura 2: Componentes e principais fluxos. O Núcleo coordena; o Planner calcula delta; o Deployer aplica via SSH, executa inspeções somente leitura quando solicitado e atualiza o *chaves_instaladas* dos servidores após cada sucesso.

O **escopo do Auditor** é estrito: o AdminForge audita as ações do Superadmin (quem alterou configuração, com que resultado). Não audita logins ou comandos dos admins nos servidores; isso é papel do *sshd* e *auditd* de cada máquina. Separadamente, o sistema oferece uma leitura operacional via SSH para inspecionar usuários locais e serviços em execução, mas essa leitura é diagnóstico, não trilha completa de atividade do servidor.

3.1 Persistência

O Store é um diretório com um arquivo YAML por entidade. Arquivos texto em vez de banco de dados: a escala (vinte admins, seiscentos servidores) cabe fácil; dá pra ler a olho nu; não precisa instalar banco; a migração futura pra modo *pull* com Git é trivial.

```

state/
|-- admins/                # um arquivo por admin

```

Tabela 1: Responsabilidades de cada componente.

Componente	O que faz
CLI	Lê o comando digitado, valida sintaxe e chama o método correspondente no Núcleo. Imprime o resultado.
Núcleo	Aplica as regras de negócio. Decide o que fazer, delega ao Store (ler/escrever), ao Planner (calcular delta) e ao Deployer (aplicar). Registra no Auditor.
Store	Persiste as entidades em arquivos YAML. Garante escrita atômica (arquivo temporário + rename) e exclusão mútua via <i>lockfile</i> .
Planner	Compara o estado desejado (grupos + permissões) com o que está em cada servidor (<i>chaves_instaladas</i>) e produz a lista de ações necessárias.
Deployer	Recebe a lista de ações e executa cada uma via SSH. Também executa comandos de inspeção somente leitura para listar usuários e serviços ativos quando o Superadmin pede auditoria operacional. Após cada sucesso de escrita, atualiza <i>chaves_instaladas</i> do servidor. Registra resultado de cada ação no Auditor.
Auditor	Persiste o histórico do que o Superadmin faz. Uma entrada por comando da CLI, com status final e lista de subações (se houver). Mantém a cadeia de <i>hashes</i> .

```
|-- admin-groups/  
|-- servers/           # cada server.yaml inclui chaves_instaladas  
|-- server-groups/  
|-- permissions.yaml  
|-- history.jsonl      # historico (append-only, uma linha por comando)  
'-- .lock              # evita escrita simultanea
```

4 Princípios de design

Duas diretrizes: os princípios *SOLID* e a ideia do *KISS* (*keep it simple, secure*). Padrões de projeto só entram quando resolvem um problema concreto; quando adicionam só indireção, ficam de fora.

4.1 KISS

O AdminForge roda quando o Superadmin digita um comando. Não tem tráfego constante, não responde a requisições simultâneas, não precisa de baixa latência. Disso vêm simplificações:

- Fluxo síncrono. Só o Deployer paraleliza, pra reduzir o tempo de aplicação em muitos servidores.
- Sem cache. Ler um YAML é barato.
- Injeção de dependências à mão. A CLI monta o grafo no início.
- Arquivos em vez de banco. Sem ACID, sem esquema, sem processo separado.
- Lockfile. Se dois comandos colidem, um falha rápido; é aceitável.

4.2 SOLID

Aplicado pragmaticamente, pelos benefícios concretos (testes mais fáceis, componentes substituíveis):

S. Responsabilidade única: cada componente da tabela anterior muda por um motivo só.

O. Aberto/fechado: trocar o Deployer SSH por Ansible é substituição, não refatoração.

L. Liskov: implementações de IStore (YAML, Git, memória) se comportam igual pro Núcleo.

I. Interfaces pequenas: Store não sabe de SSH; Deployer não lê arquivo.

D. Inversão de dependência: Núcleo depende de interfaces, não de classes concretas. Facilita testes.

4.3 Padrões aplicados

Quatro padrões, cada um resolvendo um problema concreto:

- *Repository* no Store: esconde de onde os dados vêm.
- *Strategy* no Deployer: permite trocar o mecanismo de aplicação.
- *Command* na CLI: cada comando vira um objeto, facilitando registro e validação.
- *Observer* no Auditor: Núcleo e Deployer emitem eventos; o Auditor escuta e persiste.

Deliberadamente *evitados*: Singleton, Abstract Factory, container de DI, herança profunda, cache e *async* generalizado. Nenhum desses resolve problema real nesta versão.

5 Casos de uso

O ator é o Superadmin. Cada comando da CLI corresponde a um caso de uso e gera uma entrada no Histórico. Esta seção apresenta primeiro o diagrama geral, depois o detalhamento de cada caso de uso com fluxo principal, fluxos alternativos e história de usuário correspondente.

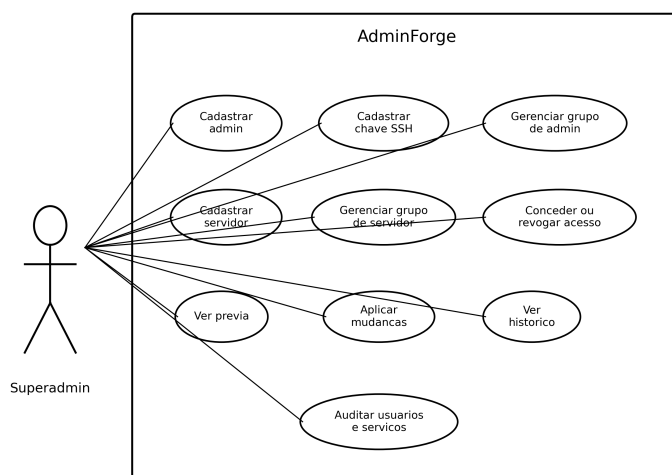


Figura 3: Casos de uso da v1, incluindo a auditoria operacional de usuários e serviços.

5.1 UC-1: Cadastrar admin

História de usuário. Como Superadmin, quero cadastrar um novo administrador no sistema, para poder, em seguida, associá-lo a grupos e conceder-lhe acessos a servidores.

Ator.

Superadmin.

Pré-condições.

CLI instalada e autenticada; nenhum admin com o mesmo *username* já existe.

Gatilho.

```
adminforge admin add <username> -nome "...--email "..."
```

Fluxo principal.

1. Superadmin digita o comando com os dados do admin.
2. CLI valida a sintaxe e chama o Núcleo.
3. Núcleo verifica que o *username* é único e que os dados obrigatórios estão presentes.
4. Núcleo escreve `admins/<username>.yaml` via Store, com status ativo.
5. Núcleo registra a Operação no Auditor com status sucesso.
6. CLI imprime confirmação.

Fluxos alternativos.

- *Username já existe:* Núcleo rejeita, Operação registrada com falha, CLI exibe erro.
- *Dados obrigatórios ausentes:* CLI rejeita antes de chamar o Núcleo.

Pós-condições.

Admin existe no Store, sem grupo e sem acesso a nenhum servidor. Operação registrada no Histórico.

5.2 UC-2: Cadastrar chave SSH

História de usuário. Como Superadmin, quero registrar a chave pública SSH de um administrador, para que, após um apply, ele possa se autenticar nos servidores aos quais tem permissão.

Ator.

Superadmin.

Pré-condições.

Admin já cadastrado (UC-1).

Gatilho.

```
adminforge key add <username> -file <arquivo.pub> ou -string «chave>"
```

Fluxo principal.

1. Superadmin fornece o caminho do arquivo `.pub` ou cola a chave.
2. Núcleo valida o formato SSH (`ssh-ed25519`, `ssh-rsa`, `ecdsa-sha2-*`) e calcula o *fingerprint*.
3. Núcleo verifica que a chave não está duplicada para o mesmo admin.
4. Núcleo registra a credencial no arquivo do admin, com status ativa.
5. Auditor registra sucesso.

Fluxos alternativos.

- *Formato inválido:* rejeitada, falha no Histórico.
- *Chave já cadastrada:* rejeitada.
- *Admin inexistente:* rejeitada.
- *Revogar:* `adminforge key revoke <fingerprint>` muda o status para revogada e, em apply seguinte, a chave é removida dos servidores.

Pós-condições.

Credencial existe e está ativa (ou revogada, conforme caso).

5.3 UC-3: Gerenciar grupo de admin

História de usuário. Como Superadmin, quero agrupar administradores por função, para poder conceder acessos por grupo em vez de um a um, reduzindo o trabalho repetitivo e o risco de esquecimento.

Ator.

Superadmin.

Pré-condições.

Para adicionar ou remover membro, o admin e o grupo devem existir.

Gatilhos.

```
adminforge group create <nome>; adminforge group add-member <grupo> <username>;  
adminforge group remove-member <grupo> <username>; adminforge group delete <nome>
```

Fluxo principal (adicionar membro).

1. Superadmin executa o comando de adição.
2. Núcleo verifica que grupo e admin existem.
3. Núcleo atualiza `admin-groups/<grupo>.yaml` acrescentando o admin.
4. Auditor registra sucesso.

Fluxos alternativos.

- *Admin já no grupo:* operação é *no-op*, registrada como sucesso no Histórico (idempotente).
- *Remover último membro:* grupo passa a existir vazio; isso é permitido.
- *Excluir grupo com permissões associadas:* bloqueado; exige remover permissões antes.

Pós-condições.

Composição do grupo atualizada. Nenhum servidor é tocado; efeito depende de apply posterior.

5.4 UC-4: Cadastrar servidor

História de usuário. Como Superadmin, quero cadastrar um servidor novo no AdminForge, para que ele passe a ser gerenciado e possa receber acessos controlados.

Ator.

Superadmin.

Pré-condições.

O servidor está acessível via SSH; a chave dedicada do AdminForge já foi instalada manualmente na conta de serviço do servidor (bootstrap, ver roadmap na Seção 8).

Gatilho.

```
adminforge server add <hostname> -ip <ipv4> -porta <22>
```

Fluxo principal.

1. Superadmin fornece endereço e porta.
2. Núcleo conecta via SSH uma vez para capturar a *host key* do servidor (TOFU, *trust on first use*).
3. Superadmin confirma o *fingerprint* exibido.
4. Núcleo grava `servers/<hostname>.yaml` com `chaves_instaladas` vazia e a *host key* salva.

5. Auditor registra sucesso.

Fluxos alternativos.

- *Servidor inacessível*: falha registrada; cadastro não é feito.
- *Hostname duplicado*: rejeitada.
- *Fingerprint recusado pelo Superadmin*: cadastro abortado.

Pós-condições.

Servidor existe, sem grupo, sem chaves instaladas, com *host key* salva para verificação estrita nas conexões seguintes.

5.5 UC-5: Gerenciar grupo de servidor

História de usuário. Como Superadmin, quero agrupar servidores por ambiente ou função, para poder conceder acessos ao conjunto em vez de um servidor por vez.

Ator.

Superadmin.

Pré-condições.

Para adicionar ou remover membro, servidor e grupo devem existir.

Gatilhos.

```
adminforge server-group create <nome>; adminforge server-group add-member <grupo>  
<hostname>; adminforge server-group remove-member <grupo> <hostname>
```

Fluxo principal (adicionar).

Análogo ao UC-3.

Fluxos alternativos.

Análogos ao UC-3; excluir grupo com permissões associadas também é bloqueado.

Pós-condições.

Composição do grupo atualizada; nenhum servidor é tocado até o próximo apply.

5.6 UC-6: Conceder ou revogar acesso

História de usuário. Como Superadmin, quero declarar que um grupo de administradores pode acessar um grupo de servidores em um determinado nível, para que o sistema propague essa decisão de forma controlada e rastreável.

Ator.

Superadmin.

Pré-condições.

Grupo de admin e grupo de servidor existem.

Gatilhos.

```
adminforge grant <grupo-admin> <grupo-servidor> -nivel shell|sudo; adminforge  
revoke <grupo-admin> <grupo-servidor>
```

Fluxo principal.

1. Superadmin executa grant.
2. Núcleo verifica que a permissão ainda não existe com o mesmo par.
3. Núcleo cria uma Permissão em `permissions.yaml`.
4. Auditor registra sucesso.

Fluxos alternativos.

- *Permissão já existe com mesmo nível:* operação idempotente.
- *Permissão existe com nível diferente:* é atualizada (sem duplicar), e o delta do próximo apply reflete a mudança.
- *Revogar permissão inexistente:* falha clara.

Pós-condições.

Estado desejado atualizado. Chaves só são efetivamente propagadas ou removidas no próximo apply.

5.7 UC-7: Ver prévia

História de usuário. Como Superadmin, quero ver exatamente o que vai mudar em cada servidor antes de aplicar, para evitar surpresas e detectar erros de configuração antes de qualquer chamada SSH.

Ator.

Superadmin.

Pré-condições.

Nenhuma específica.

Gatilho.

`adminforge preview`

Fluxo principal.

1. CLI chama o Núcleo em modo *read-only*.
2. Núcleo delega ao Planner.
3. Planner lê estado desejado (grupos + permissões) e estado aplicado (`chaves_instaladas` de cada servidor).
4. Planner produz a lista de subações necessárias (adicionar chave, remover chave).
5. CLI imprime a lista agrupada por servidor.

Fluxos alternativos.

- *Nada a fazer:* CLI informa que o estado já está sincronizado.
- *Servidor sem host key registrada:* alerta no relatório (indica cadastro incompleto).

Pós-condições.

Nenhuma mudança em nenhum servidor. Nenhuma Operação de mutação no Histórico (preview pode ser registrado como evento de leitura, opcional no M-1).

5.8 UC-8: Aplicar mudanças

História de usuário. Como Superadmin, quero efetivar o estado desejado nos servidores, propagando apenas o que mudou desde a última aplicação, para minimizar impacto, tempo de execução e risco.

Ator.

Superadmin.

Pré-condições.

Pelo menos uma diferença entre estado desejado e aplicado (caso contrário, apply é *no-op*).

Gatilho.

`adminforge apply` (ou `-yes` para pular confirmação).

Fluxo principal.

1. Núcleo abre uma Operação com status `em_andamento`.
2. Planner calcula o delta e produz a lista de subações.
3. CLI exibe a lista e pede confirmação.
4. Deployer executa as subações em paralelo, respeitando o limite configurado.
5. Para cada subação: se sucesso, atualiza `chaves_instaladas` do servidor; se falha, anota a mensagem de erro.
6. Núcleo fecha a Operação com sucesso, `sucesso_parcial` ou falha, conforme o resultado agregado.
7. CLI imprime o resumo com quantidade de sucessos e falhas.

Fluxos alternativos.

- *Servidor fora do ar*: subação em falha; entra no delta do próximo apply automaticamente.
- *Taxa de falha acima do limite (ex.: 50%)*: Deployer interrompe o lote; subações remanescentes ficam como pendentes e entram no próximo apply.
- *Host key divergente da registrada*: conexão rejeitada; subação em falha com erro claro.
- *Superadmin cancela na confirmação*: Operação encerrada como abortada, nenhuma subação executada.

Pós-condições.

Operação registrada no Histórico com lista de subações e seus status individuais. `chaves_instaladas` dos servidores com sucesso está atualizado.

*Não existe fila de “tarefas pendentes”. O sistema não guarda uma lista separada do que falhou. Pendente é sempre calculado na hora como **desejado** — **aplicado**. Se uma subação falhar, o `chaves_instaladas` do servidor não muda, então o próximo apply naturalmente a identifica no delta. Isso evita duas fontes da verdade que podem divergir.*

5.9 UC-9: Ver histórico

História de usuário. Como Superadmin, quero consultar o que foi feito no sistema, quando e por quem, para auditoria, diagnóstico de problemas e reconstituição de mudanças.

Ator.

Superadmin.

Pré-condições.

Nenhuma.

Gatilhos.

- `adminforge history list` — últimas N operações com status.
- `adminforge history show <id>` — detalhes de uma operação, incluindo lista de subações.
- `adminforge history failed` — apenas operações com falha ou `sucesso_parcial`.
- `adminforge history verify` — verifica a integridade da cadeia de *hashes*.

Fluxo principal.

CLI lê `history.jsonl` e formata em tabela.

Fluxos alternativos.

- *Cadeia quebrada*: `verify` aponta o primeiro ponto de divergência.
- *ID inexistente*: erro claro.

Pós-condições.

Nenhuma; operação de leitura.

5.10 UC-10: Auditar usuários e serviços do servidor

História de usuário. Como Superadmin, quero inspecionar quais usuários existem e quais serviços estão rodando em um servidor, para detectar incoerências operacionais, como um usuário tomcat presente sem nenhum serviço Tomcat ativo.

Ator.

Superadmin.

Pré-condições.

O servidor está cadastrado e acessível via SSH com a chave do AdminForge.

Gatilhos.

- `adminforge audit server <servidor>` — lista usuários locais e serviços ativos.
- `adminforge audit server <servidor> -user tomcat` — destaca ocorrências relacionadas a um usuário específico.
- `adminforge audit server <servidor> -service tomcat` — destaca serviços cujo nome, unidade ou processo remete ao serviço consultado.

Fluxo principal.

- CLI envia a solicitação ao Núcleo.
- Núcleo valida que o servidor existe e delega ao Deployer uma inspeção somente leitura.
- Deployer conecta via SSH e coleta: lista de usuários locais relevantes e lista de serviços em execução (por exemplo, via `systemctl` ou leitura equivalente do sistema alvo).
- Núcleo correlaciona os dados e destaca inconsistências óbvias, como usuário técnico sem serviço correspondente em execução.
- CLI imprime o relatório e o Auditor registra a operação como leitura.

Fluxos alternativos.

- *Servidor inacessível*: erro claro; operação de leitura registrada com falha.
- *Servidor sem `systemd`*: Deployer usa uma estratégia compatível com o `init` local ou informa limitação explícita.
- *Usuário encontrado sem serviço compatível*: relatório marca o achado como alerta, sem alterar estado do sistema.

Pós-condições.

Nenhuma mudança no estado desejado nem no estado aplicado; apenas diagnóstico registrado no Histórico.

6 Fluxos principais

Quatro fluxos cobrem a operação típica. Cada comando da CLI é independente e gera uma entrada própria no Histórico.

6.1 Cadastros (mudar o estado desejado)

Cadastrar admin, adicionar chave, criar grupo, associar ao grupo, conceder permissão. Todos seguem o mesmo formato: a CLI chama o Núcleo, o Núcleo valida e escreve no Store, o Auditor registra. Nenhum servidor é tocado.

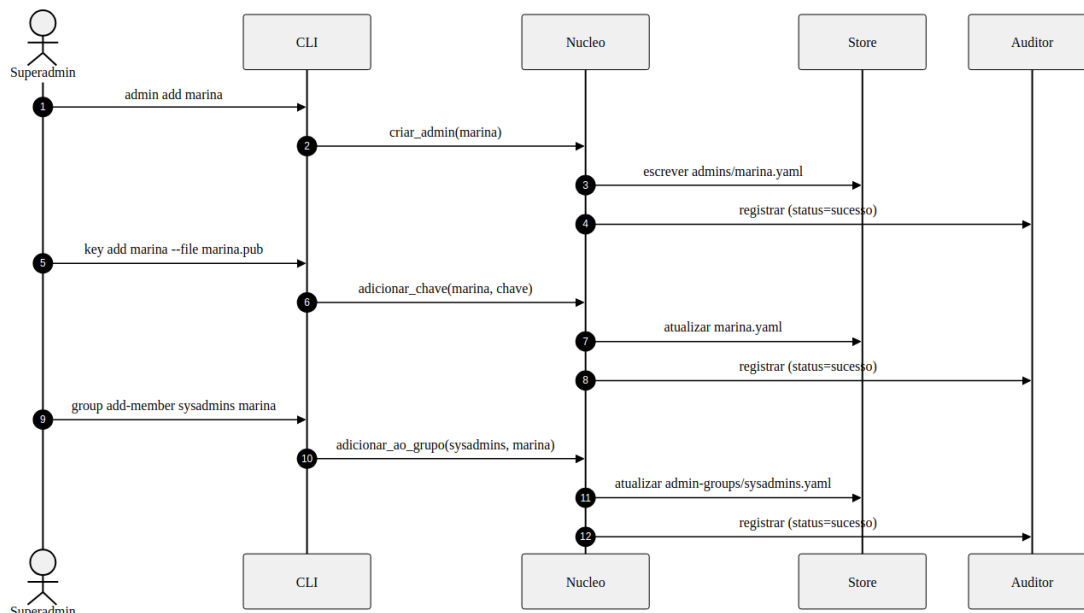


Figura 4: Três cadastros independentes. Cada um é um comando separado, com sua entrada no Histórico. Ninguém ainda tem acesso a servidor nenhum.

6.2 Aplicar mudanças

É aqui que os servidores são tocados. O Planner compara o estado desejado com o que cada servidor tem em `chaves_instaladas` e produz a lista de subações. O Deployer executa cada uma. O Histórico registra o comando e a lista de subações com status individual.

Ao final do apply, o Superadmin vê no terminal um resumo das subações. Exemplo com uma falha:

```
$ adminforge apply

Calculando plano... 3 acoes em 3 servidores.

web-01  adicionar marina  OK
web-02  adicionar marina  OK
db-03   adicionar marina  FALHA  ssh: connect timeout after 30s

Resultado: SUCESSO PARCIAL (2/3)
Operacao registrada como OP-0042.
Rode 'adminforge history show OP-0042' pra ver detalhes.
Rode 'adminforge apply' novamente pra retentar as falhas.
```

6.3 Inspeção operacional

Aqui não há mudança de estado: o Superadmin consulta um servidor para listar usuários locais e serviços ativos. O objetivo é detectar incoerências, como contas técnicas sem processo correspondente, e registrar o diagnóstico no Histórico como operação de leitura.

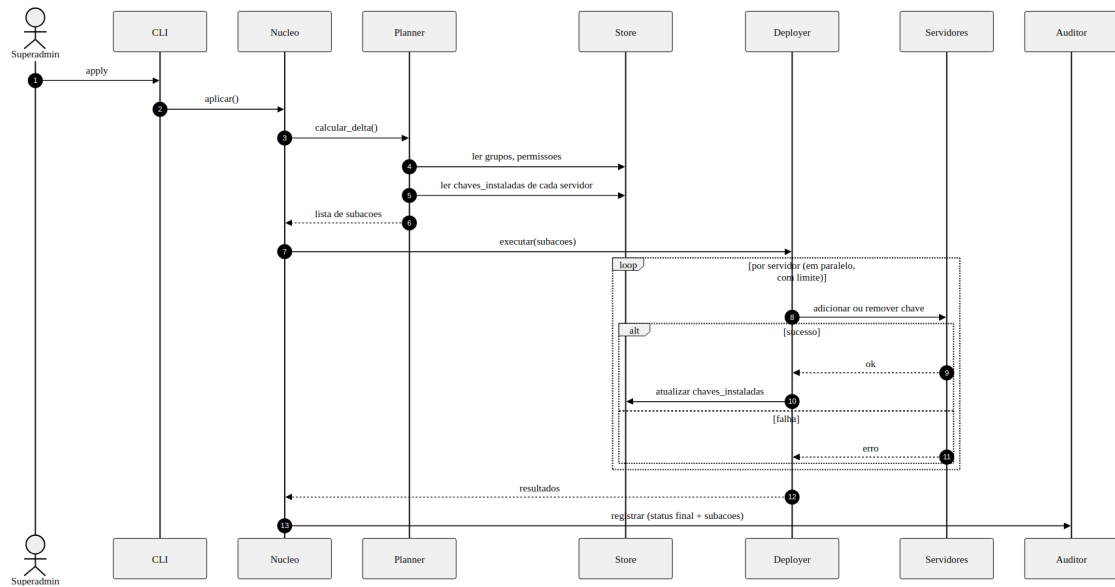


Figura 5: Apply. Se cinco chaves já estão instaladas num servidor e entra uma sexta, o Planner só gera a subação pra sexta. Subações que falharem ficam no Histórico com status próprio e entram no delta do próximo *apply*.

6.4 Desligamento

Marca o admin como inativo e revoga as credenciais. A remoção nos servidores acontece no próximo *apply*.

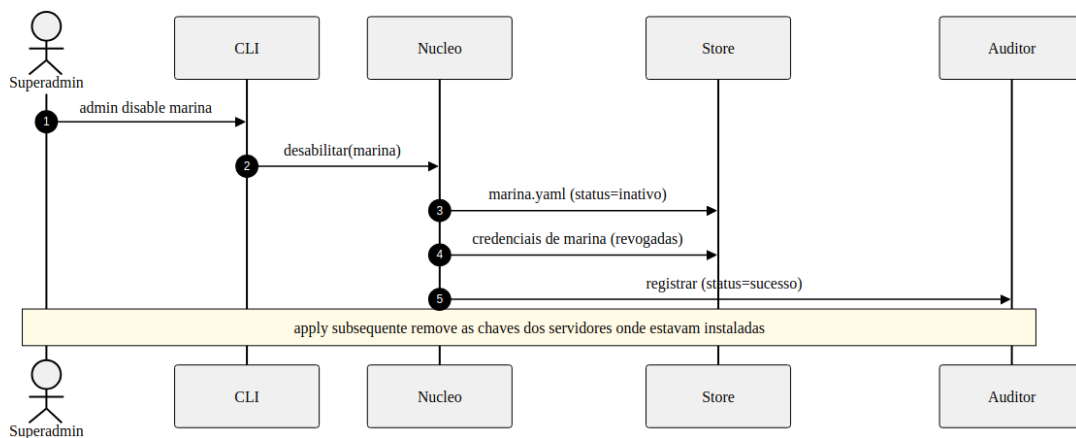


Figura 6: Desligamento. A efetivação é separada do cadastro, como todos os comandos.

Operações mudam o estado desejado; apply faz o real convergir pro desejado.

7 Cuidados de segurança

7.1 Arquivos no disco. YAMLS em diretório com permissão 0600. Cifragem seletiva com age ou sops quando necessário.

7.2 Conexão SSH. Chave dedicada ao sistema. Cada servidor tem sua *host key* salva; `StrictHostKeyChecking=no` é proibido.

7.3 Histórico imutável. Cada entrada carrega o *hash* da anterior. Alterações retroativas quebram a corrente e são detectáveis.

7.4 Escopo da auditoria. O AdminForge audita as ações do Superadmin. Logins dos admins nos servidores são registrados pelo `sshd/auditd` de cada máquina. A nova inspeção de usuários e serviços é apenas leitura operacional e não substitui auditoria nativa do host.

7.5 Limitar estrago. Apply em lotes. Se a taxa de falhas sobe, o Deployer para e deixa o restante pendente.

7.6 Desligamento garantido. Apply é idempotente. Servidores fora do ar ficam com subação em *falha* no Histórico e são retentados no próximo apply.

8 Próximos passos

M-0 Validar a modelagem (agora). Revisão com Rui e Diego.

M-1 Protótipo mínimo. Implementação em Python dos seis componentes. Comandos essenciais: cadastros, prévia, apply.

M-2 Robustez. Retentativa automática, reconciliação opcional (*apply verify* que confere `chaves_instaladas` contra o real), auditoria operacional de usuários/serviços e cifragem seletiva.

M-3 Rust e modo pull. Portar pra Rust e implementar o modo *pull*: os servidores puxam o estado de um repositório Git assinado.

8.1 Questões em aberto para a reunião

Pontos identificados durante a modelagem que precisam ser alinhados com Rui e Diego antes do M-1:

1. **Criação da conta Unix.** O AdminForge instala a chave pública em `~/username/.ssh/authorized_keys`, mas o usuário Unix precisa existir no servidor. O sistema cria a conta (`useradd`) se não existir? UID/GID são consistentes entre servidores? Se dois admins pedirem o mesmo username, o que acontece?
2. **Bootstrap de servidor novo.** Quando o Superadmin cadastra um servidor pela primeira vez, como a chave SSH do AdminForge chega lá pra permitir as conexões futuras? Precisa ser instalada manualmente uma vez? Via `cloud-init`?
3. **Rotação da chave do AdminForge.** A chave dedicada ao sistema eventualmente vai precisar ser trocada. Como é feito sem perder acesso à frota?
4. **Nível sudo.** Onde ele é aplicado? Via `/etc/sudoers.d/adminforge-<admin>?` Isso define se o Deployer precisa escrever em dois lugares por subação.
5. **Descomissionamento de servidor.** Qual o comando? O sistema tenta limpar as chaves antes, ou só remove o registro?
6. **Backup do diretório state/.** Se perder, o sistema continua funcionando (a reconciliação redescobre o estado real), mas o histórico some. Vale um `.tar.gz` por dia? Git desde o M-1?
7. **Critério de correlação usuário-serviço.** Como o sistema decide que um usuário técnico tomcat deveria ter um serviço correspondente? Por nome da unidade no `systemd`, por processo dono do PID, por configuração declarada ou por uma tabela de mapeamentos mantida no Store?